



파머완 4주차 내용정리

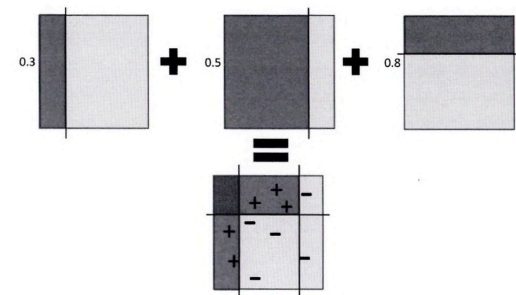
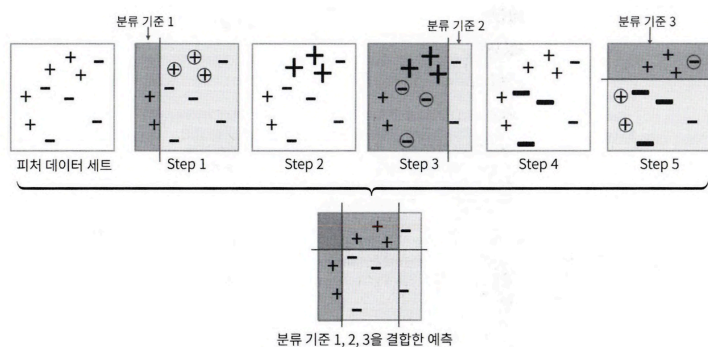
▼ Chapter. 4 분류

▼ 5. GBM(Gradient Boosting Machine)

▼ GBM의 개요 및 실습

부스팅 알고리즘: 여러 개의 약한 학습기를 순차적으로 학습-예측하면서 잘못 예측한 데이터에 가중치 부여를 통해 오류를 개선해 나가면서 학습하는 방식

- AdaBoost: 오류 데이터에 가중치를 부여하면서 부스팅을 수행하는 대표적인 알고리즘
각각 오류에 가중치를 부여한 후 모두 결합해 예측 수행



- 그래디언트 부스트: 가중치 업데이트를 경사 하강법(Gradient Descent)을 이용
오류식: $h(x) = y - F(x)$
 - y = 분류의 실제 결과값
 - $F(x)$ = 피쳐에 기반한 예측 함수
 - 오류식을 최소화하는 방향성을 가지고 반복적으로 가중치 값 업데이트수행시간 오래걸림

▼ GBM 하이퍼 파라미터 소개

- loss: 경사 하강법에서 사용할 비용 함수 지정. 기본값='deviance'
- learning_rate: GBM이 학습을 진행할 때마다 적용하는 학습률. weak learner가 순차적으로 오류 값을 보정해 나가는 데 적용하는 계수. 0~1 사이의 값, 기본값=0.1
- n_estimators: weak learner의 개수
- subsample: weak learner가 학습에 사용하는 데이터의 샘플링 비율. 기본값=1

▼ 6. XGBoost(eXtra Gradient Boost)

▼ XGBoost 개요

장점

- 뛰어난 예측 성능
- 빠른 수행 시간
- 과적합 규제
- Tree pruning: 더 이상 긍정 이득이 없는 분할을 가지치기 해서 분할 수를 줄임
- 자체 내장된 교차 검증
- 결손값 자체 처리

▼ 파이썬 래퍼 XGBoost 하이퍼 파라미터

- 일반 파라미터: 스레드의 개수나 silent 모드 등의 선택을 위한 파라미터
 - booster: gbtree or gblinear
 - silent: default=0
 - nthread: CPU 실행 스레드 개수 조정
- 부스터 파라미터: 트리 최적화, 부스팅, regularization 등과 관련 파라미터
 - eta: GBM의 learning rate와 같은 파라미터
 - num_boost_rounds: GBM의 n_estimators와 같은 파라미터
 - min_child_weight: 트리에서 추가적으로 가지를 나눌지를 결정하기 위해 필요한 데이터들의 weight 총합
 - gamma: 트리의 리프 노드를 추가적으로 나눌지를 결정할 최소 손실 감소 값
 - max_depth: 트리 기반 알고리즘의 max_depth와 같음
 - sub_sample: GBM의 subsample과 같음
 - colsample_bytree: GBM의 max_features와 유사
 - lambda: L2 Regularization 적용 값. 값이 클수록 과적합 감소
 - alpha: L1 Regularization 적용 값. 값이 클수록 과적합 감소
 - scale_pos_weight: 특정 값으로 치우친 비대칭한 클래스로 구성된 데이터 세트의 균형을 유지하기 위한 파라미터
- 학습 태스크 파라미터: 학습 수행 시의 객체 함수, 평가를 위한 지표 등을 설정하는 파라미터
 - objective: 최솟값을 가져야 할 손실 함수 정의
 - binary:logistic: 이진 분류일 때 적용
 - multi:softmax: 다중 분류일 때 적용
 - multi:softprob: multi:softmax와 유사하나 개별 레이블 클래스의 해당되는 예측 확률을 반환
 - eval_metric: 검증에 사용되는 함수를 정의
- 과적합 문제
 - eta 값, max_depth 낮춤
 - min_child_weight, gamma 값 높임
 - subsample, colsample_bytree 조정
- Early Stopping: n_estimators에 지정한 부스팅 반복 횟수에 도달하지 않아도 예측 오류가 더 이상 개선되지 않으면 반복을 끝까지 수행하지 않고 중지 → 수행 시간 개선
- DMatrix: XGBoost 전용 데이터 객체. data, label 입력
- 시각화 기능
 - plot_importance(): f스코어를 기반으로 피처의 중요도를 막대그래프 형식으로 나타냄
 - f스코어: 해당 피처가 트리 분할 시 얼마나 자주 사용되었는지를 지표로 나타낸 값
 - xgboost.to_graphviz(): 규칙 트리 구조 그릴 수 있음
- cv(): 교차 검증 수행 후 최적의 파라미터 구함

```
xgboost.cv(params, dtrain, num_boost_round=10, nfold=3, stratified=False, folds=None, metrics=(),
obj=None, feval=None, maximize=False, early_stopping_rounds=None, fpreproc=None, as_pandas=True,
verbose_eval=None, show_stdv=True, seed=0, callbacks=None, shuffle=True)
```

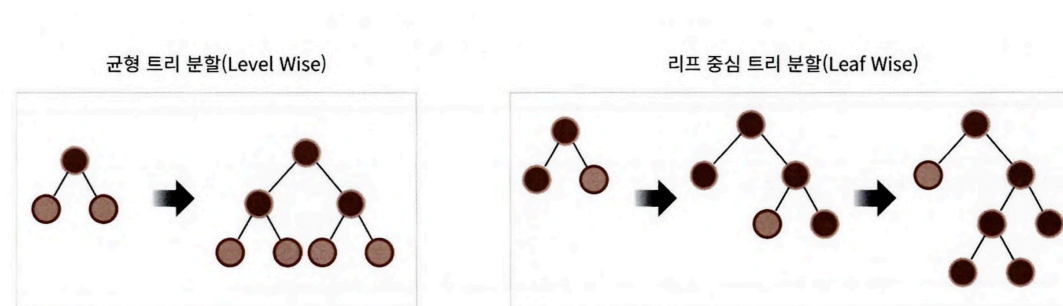
▼ 사이킷런 래퍼 XGBoost의 개요 및 적용

- XGBClassifier / XGBRegressor
- 네이티브 하이퍼 파라미터 변경
 - eta → learning_rate

- sub_sample → subsample
- lambda → reg_lambda
- alpha → reg_alpha

▼ 7. LightGBM

- 장점
 - 더 빠른 학습과 예측 수행 시간
 - 더 작은 메모리 사용량
 - 카테고리형 피처의 자동 변환과 최적 분할
- 단점
 - 적은 데이터 세트(<10,000건)에 적용할 경우 과적합이 발생하기 쉬움
- 리프 중심 트리 분할: 균형을 맞추지 않고, 최대 손실 값을 가지는 리프 노드를 지속적으로 분할
→ 트리의 깊이가 깊어지고 비대칭적 → 반복할수록 결국 예측 오류 손실 최소화



▼ LightGBM 하이퍼 파라미터

▼ 주요 파라미터

- num_iterations: 반복 수행하려는 트리의 개수 지정
- learning_rate: 0~1, 부스팅 스텝을 반복적으로 수행할 때 업데이트되는 학습률 값
- max_depth: 트리 깊이 제한
- min_data_in_leaf: 결정 트리의 min_samples_leaf와 같음(=min_child_samples)
- num_leaves: 하나의 트리가 가질 수 있는 최대 리프 개수
- boosting: 부스팅의 트리를 생성하는 알고리즘을 기술. gbd, rf
- bagging_fraction: 과적합을 방지하기 위해 데이터를 샘플링하는 비율 지정
- feature_fraction: 개별 트리를 학습할 때마다 무작위로 선택하는 피처의 비율
- lambda_l2: L2 regulation 제어를 위한 값
- lambda_l1: L1 regulation 제어를 위한 값

▼ Learning Task 파라미터

- objective: 최솟값을 가져야 할 손실함수 정의

▼ 하이퍼 파라미터 튜닝 방안

num_leaves의 개수를 중심으로 min_child_samples와 max_depth를 함께 조정

- num_leaves: 높이면 정확도가 높아지지만 모델이 복잡해져 과적합 영향도가 커짐
- min_child_samples: 높이면 트리가 깊어지는 것 방지
- max_depth: 깊이의 크기를 제한
- learning_rate 작게 하면서 n_estimators 크게

▼ 파이썬 래퍼 LightGBM과 사이킷런 래퍼 XGBoost, LightGBM 하이퍼 파라미터 비교

유형	파이썬 래퍼 LightGBM	사이킷런 래퍼 LightGBM	사이킷런 래퍼 XGBoost
파라미터명	num_iterations	n_estimators	n_estimators
	learning_rate	learning_rate	learning_rate
	max_depth	max_depth	max_depth
	min_data_in_leaf	min_child_samples	N/A
	bagging_fraction	subsample	subsample
	feature_fraction	colsample_bytree	colsample_bytree
	lambda_l2	reg_lambda	reg_lambda
	lambda_l1	reg_alpha	reg_alpha
	early_stopping_round	early_stopping_rounds	early_stopping_rounds
	num_leaves	num_leaves	N/A
	min_sum_hessian_in_leaf	min_child_weight	min_child_weight

▼ 8. 베이지안 최적화 기반의 HyperOpt를 이용한 하이퍼 파라미터 튜닝

Grid Search 방식은 튜닝해야 할 하이퍼 파라미터 개수가 많을 경우 최적화 수행 시간이 오래 걸림

XGBoost, LightGBM - 하이퍼 파라미터 개수가 다른 알고리즘에 비해서 많음

→ 베이지안 최적화 기법 사용!

▼ 베이지안 최적화 개요

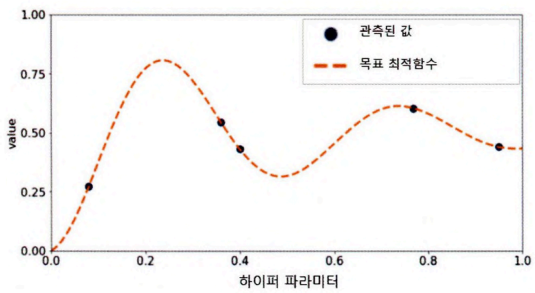
베이지안 최적화: 목적 함수 식을 제대로 알 수 없는 블랙 박스 형태의 함수에서 최대 또는 최소 함수 반환 값을 만드는 최적 입력값을 가능한 적은 시도를 통해 빠르고 효과적으로 찾아주는 방식

베이지안 확률에 기반을 둠. 새로운 데이터를 입력 받았을 때 최적 함수를 예측하는 사후 모델을 개선

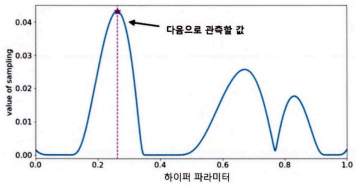
대체 모델(Surrogate Model): 획득 함수로부터 최적 함수를 예측할 수 있는 입력값을 추천 받은 뒤 이를 기반으로 최적 함수 모델 개선

획득 함수(Acquisition Function): 개선된 대체 모델을 기반으로 최적 입력값 계산

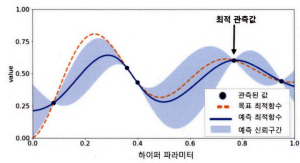
- Step 1: 최초에는 랜덤하게 하이퍼 파라미터들을 샘플링하고 성능 결과를 관측합니다. 아래 그림에서 검은색 원은 특정 하이퍼 파라미터가 입력되었을 때 관측된 성능 지표 결과값을 뜻하며 주황색 사선은 찾아야 할 목표 최적함수입니다.



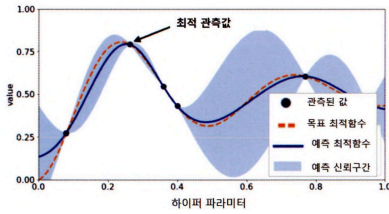
- Step 3: 추정된 최적 함수를 기반으로 획득 함수(Acquisition Function)는 다음으로 관측할 하이퍼 파라미터 값을 계산합니다. 획득 함수는 이전의 최적 관측값보다 더 큰 최솟값을 가질 가능성이 높은 지점을 찾아서 다음에 관측할 하이퍼 파라미터를 대체 모델에 전달합니다.



- Step 2: 관측된 값을 기반으로 대체 모델은 최적 함수를 추정합니다. 아래 그림에서 파란색 실선은 대체 모델이 추정한 최적 함수입니다. 넓은 파란색으로 되어 있는 영역은 예측된 함수의 신뢰 구간입니다. 추정된 함수의 값과 값 오류 편차를 의미하여 추정 함수의 불확실성을 나타냅니다. 최적 관측값은 y축 value에서 가장 높은 값을 가질 때의 하이퍼 파라미터입니다.



- Step 4: 획득 함수로부터 전달된 하이퍼 파라미터를 수행하여 관측된 값을 기반으로 대체 모델은 갱신되어 다시 최적 함수를 예측 추정합니다.



최적 함수를 추정할 때 일반적으로 가우시안 프로세스 적용

▼ HyperOpt 사용하기

1. 입력 변수명과 입력값의 검색 공간 설정
2. 목적 함수의 설정
3. 목적 함수의 반환 최솟값을 가지는 최적 입력값을 유추

- 입력값의 검색 공간을 제공하는 대표적인 함수

- `hp.quniform(label, low, high, q)`: label로 지정된 입력값 변수 검색 공간을 최솟값 low에서 최댓값 high까지 q의 간격을 가지고 설정.
- `hp.uniform(label, low, high)`: 최솟값 low에서 최댓값 high까지 정규 분포 형태의 검색 공간 설정
- `hp.randint(label, upper)`: 0부터 최댓값 upper까지 random한 정숫값으로 검색 공간 설정.
- `hp.loguniform(label, low, high)`: `exp(uniform(low, high))`값을 반환하며, 반환 값의 log 변환 된 값은 정규 분포 형태를 가지는 검색 공간 설정.
- `hp.choice(label, options)`: 검색 값이 문자열 또는 문자열과 숫자값이 섞여 있을 경우 설정. Options는 리스트나 튜플 형태로 제공되며 `hp.choice('tree_criterion', ['gini', 'entropy'])`과 같이 설정하면 입력 변수 `tree_criterion`의 값을 'gini'와 'entropy'로 설정하여 입력함.

- 목적 함수는 반드시 변숫값과 검색 공간을 가지는 딕셔너리를 인자로 받아야함
- `fmin()` 함수: 목적 함수의 반환값이 최소가 될 수 있는 최적의 입력값을 베이지안 최적화 기법에 기반하여 찾음
 - `fn`: `objective_func`와 같은 목적 함수
 - `space`: `search_space`와 같은 검색 공간 딕셔너리
 - `algo`: 베이지안 최적화 적용 알고리즘
 - `max_evals`: 최적 입력값을 찾기 위한 입력값 시도 횟수
 - `trials`: 최적 입력값을 찾기 위해 시도한 입력값 및 해당 입력값의 목적 함수 반환값 결과를 저장
 - `state`: 랜덤 시드 값

▼ HyperOpts를 이용한 XGBoost 하이퍼 파라미터 최적화

목적 함수 설정 시 주의점

- 특정 하이퍼 파라미터들은 정숫값만 입력 받는데 HyperOpt는 입력값과 반환값이 모두 실수형이기 때문에 하이퍼 파라미터 입력 시 형변환을 해줘야 함
- HyperOpt의 목적 함수는 최솟값을 반환할 수 있도록 최적화해야 하기 때문에 성능 값이 클수록 좋은 성능 지표일 경우 -1을 곱해줘야 함

▼ 10. 분류 실습 - 캐글 신용카드 사기 검출

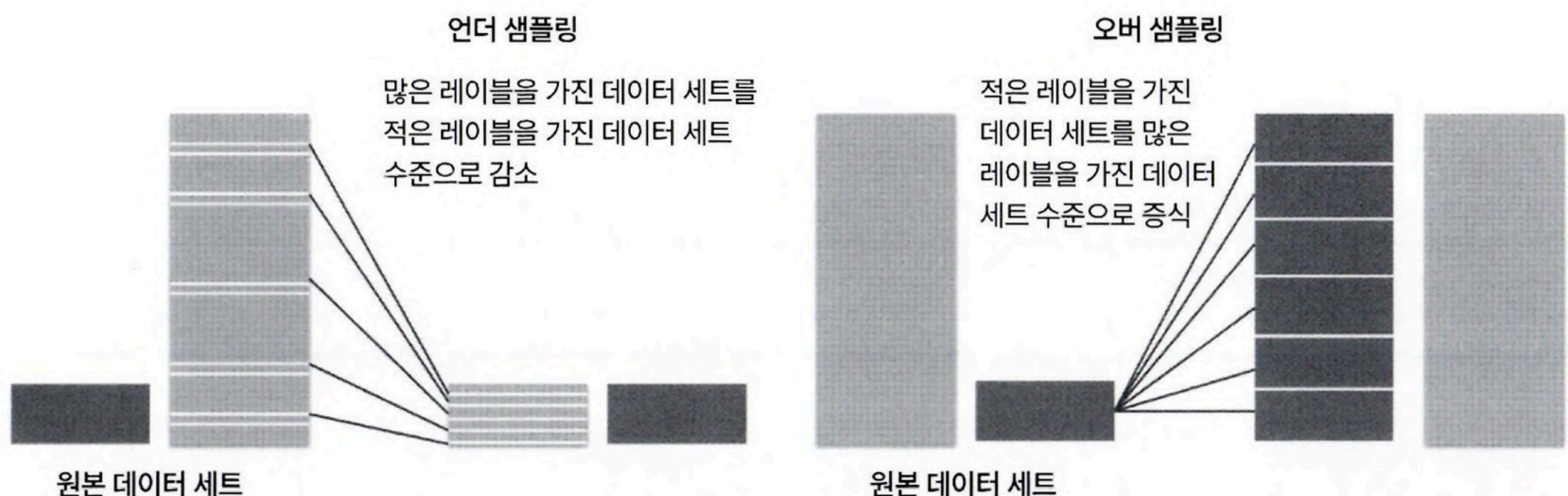
▼ 언더 샘플링과 오버 샘플링의 이해

이상 레이블 <<< 정상 레이블

정상 레이블로 치우친 학습으로 이상 데이터 검출이 어려워짐

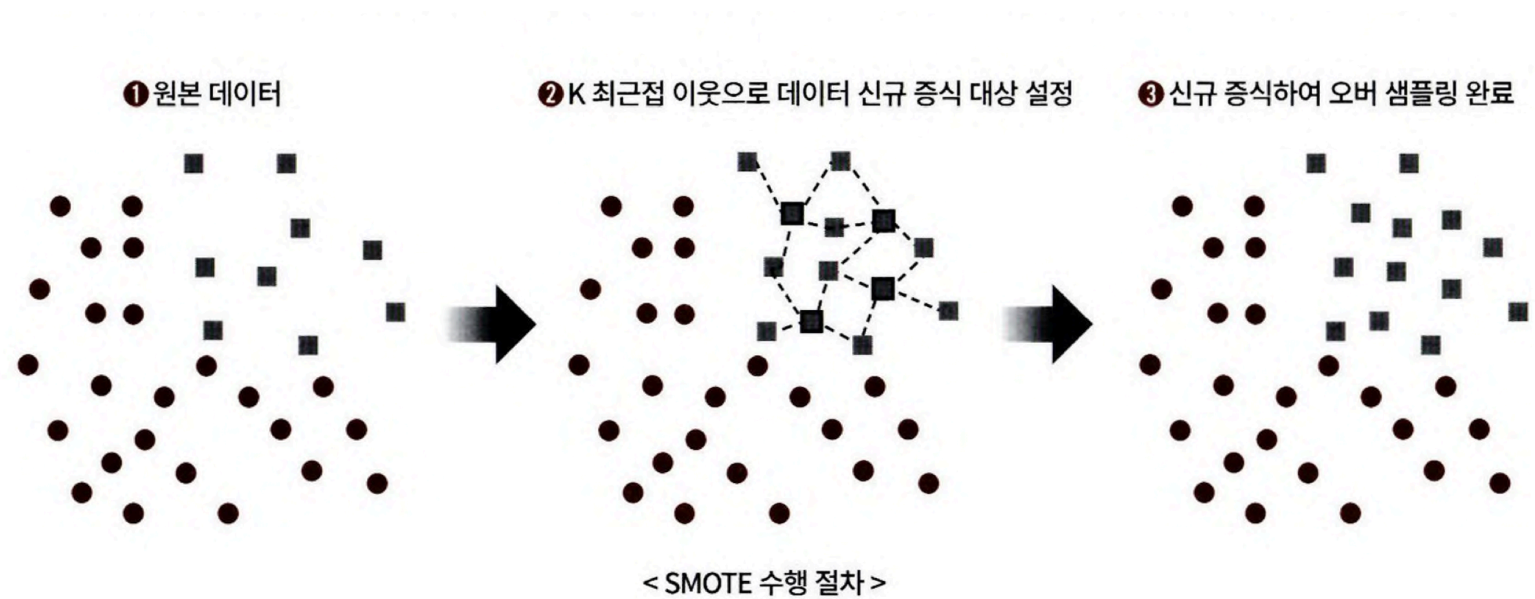
적절한 학습 데이터를 확보해야 함

→ 언더 샘플링 & 오버 샘플링



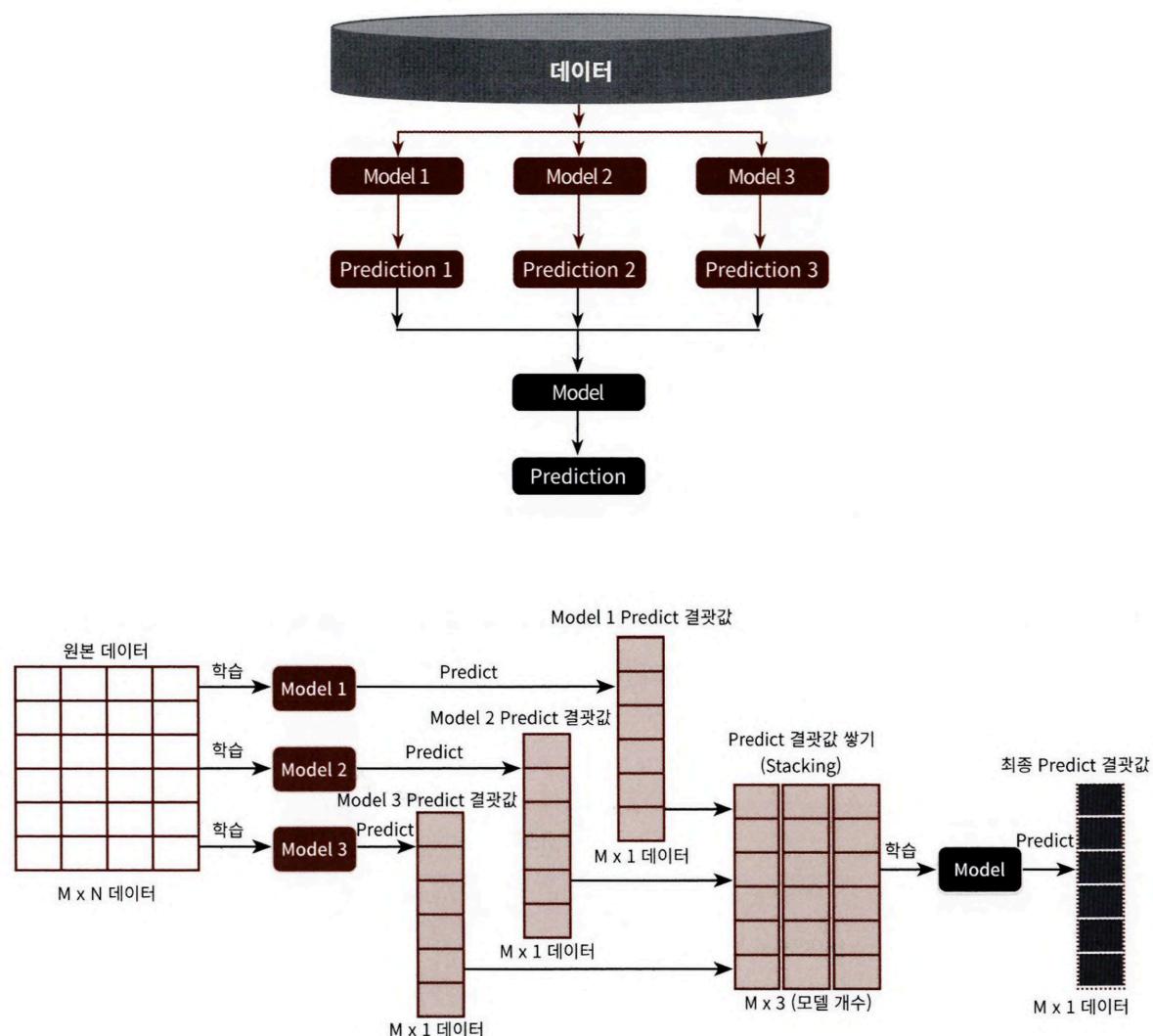
- 언더 샘플링: 많은 데이터 세트를 적은 데이터 세트 수준으로 감소시키는 방식. 정상 레이블 데이터를 이상 레이블 데이터 수준으로 줄임.

- **오버 샘플링**: 이상 데이터와 같이 적은 데이터 세트를 증식하여 학습을 위한 충분한 데이터를 확보하는 방법. 원본 데이터의 피쳐 값들을 아주 약간만 변경.
 - **SMOTE(Synthetic Minority Over-sampling Technique)**



▼ 11. 스택킹 앙상블

- **스태킹**: 개별 알고리즘의 예측 결과 데이터 세트를 최종적인 메타 데이터 세트로 만들어 별도의 ML 알고리즘으로 최종 학습을 수행하고 테스트 데이터를 기반으로 다시 최종 예측을 수행하는 방식
- 기반 모델 → 메타 모델
- 핵심: 여러 개별 모델의 예측 데이터를 각각 스택킹 형태로 결합해 최종 메타 모델의 학습용 피쳐 데이터 세트와 테스트용 피쳐 데이터 세트를 만드는 것



→ 모델별로 도출된 예측 레이블 값을 다시 합해서 새로운 데이터 세트를 만들고, 스택킹된 데이터 세트에 대해 최종 모델을 적용해 최종 예측을 하는 것!

▼ 기본 스택킹 모델

- 모델링 예시
- 개별 모델: KNN, 랜덤 포레스트, 결정 트리, 에이다부스트

▼ CV 세트 기반의 스택킹

과적합을 개선하기 위해 교차 검증 기반으로 예측된 결과 데이터를 이용

1. 각 모델별로 원본 학습/테스트 데이터를 예측한 결과 값을 기반으로 메타 모델을 위한 학습용/테스트용 데이터를 생성
2. 1에서 개별 모델들이 생성한 학습용 데이터를 모두 스택킹 형태로 합쳐서 메타 모델이 학습할 최종 학습용 데이터 세트를 생성.
각 모델들이 생성한 테스트용 데이터도 모두 스택킹 형태로 합쳐서 메타 모델이 예측할 최종 데이터 세트를 생성.

메타 모델은 최종적으로 생성된 학습 데이터 세트와 원본 학습 데이터의 레이블 데이터를 기반으로 학습 → 최종적으로 생성된 테스트 데이터 세트를 예측 → 원본 테스트 데이터의 레이블 데이터를 기반으로 평가

